Exercise 2: Load Data + Time Travel

Objective

Load the Airbnb dataset into your `AIRBNB.DEV` schema, then use Snowflake's **Time Travel** to recover data from past points in time.

Prerequisites

- Exercise 1 completed (`WH_DBT_DEV` warehouse, `AIRBNB.DEV` schema, `TRANSFORM_ROLE` and `dbt_user` all created)

Step 1: Set your context

Open a new worksheet (or reuse the one from Ex1). Ensure the context is correct:

```
USE ROLE ACCOUNTADMIN;
USE WAREHOUSE WH_DBT_DEV;
USE DATABASE AIRBNB;
USE SCHEMA DEV;
```

Step 2: Create the raw tables

We will load the same Airbnb Barcelona dataset used in S10. Run each of the three blocks below:

```
CREATE OR REPLACE TABLE raw_listings (
    id             INT,
    name           STRING,
    host_id        INT,
    neighbourhood  STRING,
    room_type      STRING,
    minimum_nights INT,
    price          INT,
    created_at     DATE,
    updated_at     DATE
);

INSERT INTO raw_listings VALUES
  (1001, 'Cozy Studio in Gracia',          101, 'Gracia',       'Entire home/apt',
2,  85, '2024-01-01', '2025-01-15'),
  (1002, 'Sunny Room near Park',           102, 'Gracia',       'Private room',
1,  45, '2024-01-02', '2025-02-20'),
  (1003, 'Modern Flat in Eixample',        101, 'Eixample',     'Entire home/apt',
3, 120, '2024-01-03', '2025-01-10'),
  (1004, 'Charming Apartment Gothic',      103, 'Barri Gotic',  'Entire home/apt',
2,  95, '2024-01-04', '2025-03-01'),
  (1005, 'Beachfront Studio Barceloneta',  102, 'Barceloneta',  'Entire home/apt',
1, 110, '2024-01-05', '2025-02-01'),
  (1006, 'Loft in Poblenou',               104, 'Poblenou',     'Entire home/apt',
```

```
3,  75, '2024-01-06', '2025-01-20'),
  (1007, 'Budget Room Sagrada Familia',    103, 'Eixample',     'Private room',
1,  35, '2024-01-07', '2025-03-15'),
  (1008, 'Penthouse with Terrace',         104, 'Eixample',     'Entire home/apt',
2, 200, '2024-01-08', '2025-01-05');
```

```sql
CREATE OR REPLACE TABLE raw_hosts (
    id            INT,
    name          STRING,
    is_superhost  STRING,
    created_at    DATE,
    updated_at    DATE
);

INSERT INTO raw_hosts VALUES
  (101, 'Maria Garcia',    't', '2023-01-01', '2025-01-10'),
  (102, 'Carlos Lopez',    'f', '2023-02-15', '2025-02-20'),
  (103, 'Ana Fernandez',   't', '2023-03-01', '2025-03-01'),
  (104, 'Juan Martinez',   'f', '2023-04-10', '2025-01-25');
```

```sql
CREATE OR REPLACE TABLE raw_reviews (
    listing_id     INT,
    reviewer_name  STRING,
    rating         INT,
    review_date    DATE
);

INSERT INTO raw_reviews VALUES
  (1001, 'John Smith',    5, '2025-01-20'),
  (1001, 'Sarah Jones',   4, '2025-02-10'),
  (1002, 'Mike Brown',    5, '2025-01-15'),
  (1003, 'Emma Wilson',   3, '2025-02-05'),
  (1004, 'Tom Davis',     5, '2025-03-05'),
  (1005, 'Alice Lee',     4, '2025-01-30'),
  (1006, 'Bob Taylor',    5, '2025-02-15'),
  (1007, 'Clara Wright',  4, '2025-03-10'),
  (1008, 'David Martin',  5, '2025-01-08'),
  (1001, 'Pierre Dupont', 5, '2025-03-01');
```

Verify the row counts:

```sql
SELECT 'raw_listings' AS table_name, COUNT(*) AS rows FROM raw_listings
UNION ALL
SELECT 'raw_hosts',    COUNT(*) FROM raw_hosts
UNION ALL
SELECT 'raw_reviews',  COUNT(*) FROM raw_reviews;
```

Expected:

TABLE_NAME	ROWS
raw_listings	8
raw_hosts	4
raw_reviews	10

> ***In production*** *data would not be loaded with inline* `INSERT`*. You would use* ***stages*** *(pointing to S3/Azure Blob/GCS) +* `COPY INTO` *for bulk loads, or* ***Snowpipe*** *for continuous ingestion. We use inline INSERT here for classroom simplicity.*

About Time Travel

Snowflake stores data in **immutable micro-partitions**. When you delete or update rows, the old micro-partitions are kept for a retention period (1 day on the default edition, up to 90 days on Enterprise).

This means you can **query or restore data as it existed at any point in the recent past** — no backups needed.

Step 3: Accidentally delete some data

Simulate a mistake — someone runs a `DELETE` without thinking:

```
DELETE FROM raw_listings WHERE price > 100;
```

Check what remains:

```
SELECT COUNT(*) AS remaining FROM raw_listings;
```

Expected: **5 rows** (the 3 listings with price > 100 were deleted).

```
SELECT id, name, price FROM raw_listings ORDER BY price DESC;
```

The Penthouse ($200), Modern Flat ($120) and Beachfront Studio ($110) are gone.

Step 4: Query the data BEFORE the delete

The `AT(OFFSET => -N)` clause reads the table as it existed N seconds ago:

```
-- Query the table as it was 5 minutes ago
SELECT COUNT(*) AS total_before_delete
FROM raw_listings AT(OFFSET => -300);
```

Expected: **8 rows** — all the data is still there in the historical version.

See the deleted rows explicitly:

```sql
SELECT id, name, price
FROM raw_listings AT(OFFSET => -300)
WHERE price > 100
ORDER BY price DESC;
```

You can see the Penthouse, Modern Flat and Beachfront Studio — even though they no longer exist in the current table.

> ***How it works**: Snowflake reads the micro-partitions that were active 300 seconds ago. The old partitions are still on disk — they just are not visible to normal queries until the retention period expires.*

Step 5: Restore the deleted data

Re-insert the missing rows from the historical version:

```sql
INSERT INTO raw_listings
  SELECT * FROM raw_listings AT(OFFSET => -300)
  WHERE id NOT IN (SELECT id FROM raw_listings);
```

Verify:

```sql
SELECT COUNT(*) AS total_after_restore FROM raw_listings;
```

Expected: **8 rows** — all data is back.

Step 6: Time Travel with DROP + UNDROP

Time Travel also works with whole tables:

```sql
-- Create a throwaway table
CREATE TABLE test_drop AS SELECT * FROM raw_hosts;
SELECT COUNT(*) FROM test_drop;  -- 4
```

Drop it:

```sql
DROP TABLE test_drop;
```

Verify the table is gone:

```sql
-- This fails: "Object 'TEST_DROP' does not exist"
SELECT COUNT(*) FROM test_drop;
```

Recover with UNDROP:

```sql
UNDROP TABLE test_drop;
SELECT COUNT(*) FROM test_drop;  -- 4, back
```

Clean up:

```
DROP TABLE test_drop;
```

Checklist

- [ ] `raw_listings` (8 rows), `raw_hosts` (4 rows), `raw_reviews` (10 rows) exist
- [ ] You executed a DELETE, queried the data with `AT(OFFSET => -N)`, and restored it
- [ ] You dropped a table and recovered it with `UNDROP`

Key Concepts

Concept	What it means
Time Travel	Query or restore data as it existed in the recent past
`AT(OFFSET => -N)`	Go back N seconds from now
`BEFORE(STATEMENT => '...')`	Go back to just before a specific query ID
`UNDROP TABLE`	Restore a dropped table within the retention period
Retention period	1 day by default, up to 90 days on Enterprise edition

---

Questions

1. If the Time Travel retention is 1 day and you deleted data 2 days ago, can you still recover it?
2. Why are micro-partitions **immutable**? (Hint: what would happen to Time Travel if they were mutable?)
3. A developer accidentally ran `DELETE FROM raw_listings` without a `WHERE` clause and then ran more queries for 15 minutes before noticing. How would you recover the data?